CS683 Course Project: Page-Cross Prefetching at the L1 Data Cache

Checkpoint I Report

Sailesh Kumar Sahoo 24B1208

1 Introduction

1.1 Problem

- The L1 data cache (L1D) is virtually indexed and physically tagged (VIPT). The L1D prefetcher works on virtual addresses.
- So the L1D prefetcher can predict addresses in the next 4 KB page. This is a page-cross prefetch.
- A page-cross prefetch needs the translation of the new page before it can go to memory. If the translation is not in the Translation Lookaside Buffers (TLBs), a page walk starts for a prefetch that may be useless.
- Most academic L1D prefetchers discard page-cross prefetches. Recent work shows that allowing all of them helps some workloads and hurts others [2].

1.2 Possible costs of a page-cross prefetch

- Wrong prediction: wasted cache space and bandwidth.
- Pollution of the second-level TLB (STLB): prefetch translations evict translations that demand loads need.
- Speculative page walks: extra memory accesses in L2 and the last-level cache (LLC).
- Late prefetches: a correct prefetch waits for a page walk and arrives too late.

1.3 Idea

- Find out whether page-crossing hurts because of wrong predictions or because of translation side effects.
- Reduce the translation side effects with a small Translation Prefetch Buffer (tPB) beside the STLB. The tPB holds only translations fetched by L1D page-cross prefetches, so they do not pollute the STLB.
- IP-CaT [3] proposes the tPB for instruction prefetches only. I want to apply it to data prefetches, and see how it performs there.

2 Background

- **Address translation.** A virtual address has a virtual page number (VPN) and a page offset. The data TLB (DTLB) and the STLB cache VPN to physical page number (PPN) translations. On an STLB miss, the page table walker reads the page table from memory.
- **VIPT L1D.** The set index comes from the page offset bits, which are the same in the virtual and physical address. The tag comes from the physical address after translation.

- **Why only the L1D prefetcher can cross pages.** L2 and LLC prefetchers see only physical addresses. The next physical page has no relation to the program's next virtual page, so they must stop at the page boundary. The L1D prefetcher sees virtual addresses, so its next page is the program's own next page. It only needs a translation.
- **Prior work.**
  - Berti [1]: accurate L1D prefetcher using per-instruction local deltas on virtual addresses. Usually evaluated with page-cross prefetches discarded.
  - MOKA and DRIPPER [2]: a perceptron filter that decides which L1D page-cross prefetches to issue. Allowed prefetches still insert their translations into the DTLB and STLB.
  - IP-CaT [3]: tPB for page-cross instruction prefetches. Not evaluated for data.
  - ATP and SBFP [4]: TLB prefetching with a separate buffer for prefetched translations. Targets demand TLB misses, not translations for cache prefetches.

3 Checkpoint I Work

3.1 Code study of the PA2 ChampSim

- L1D prefetches carry virtual addresses. Each prefetch waits in the L1D prefetch queue until it is translated.
- L1D prefetches skip the DTLB: every prefetch, same-page or page-cross, sends its translation request directly to the STLB.
- The simulator does not drop page-cross prefetches. The prefetcher decides. The given IP-stride prefetcher stops at the page boundary.
- The simulator already prints the counters needed: page-cross and same-page prefetch requests, and STLB accesses and misses for L1D prefetch translations.

3.2 IP-stride prefetcher (given in PA2)

- A table indexed by the load instruction address (PC) stores the last cache line and the last stride for each PC.
- If the same stride repeats, it prefetches the next 3 lines along the stride (degree 3).
- The original version stops when a predicted line is in a different 4 KB page.

3.3 Changes

- `ip_stride_permit.l1d_pref`: copy of `ip_stride.l1d_pref` with the page-boundary check removed (4 lines). No other simulator file changes.
- Two binaries: Discard (original IP-stride) and Permit (IP-stride without the check).
- Degree variants for the sweep (Section 4.4): copies of both files with only `PREFETCH_DEGREE` changed (8, 16, 32).
- Free translation (Section 4.6): the simulator's existing option `NO_TRANSLATION_PENALTY_FOR_PREFETCHES` (one commented line in `src/cache.cc`) enabled only for two extra binaries.
- Helper scripts: `run_all.sh` runs all traces with both binaries in parallel. `parse_results.py` collects the printed counters into one table.

4 Results

4.1 Setup

- Simulator: ChampSim from the CS683 PA2 repository, built with GCC 7.5 on Ubuntu (WSL).
- Core: 1 core, hashed perceptron branch predictor.

- Translation: DTLB 64 entries (16 sets, 4 ways), STLB 1536 entries (128 sets, 12 ways), 4 KB pages.
- Replacement: LRU in all caches and TLBs. L1D prefetcher: IP-stride. No other prefetchers.
- Workloads:
  - The 3 traces provided with PA2 (Trace1, Trace2, Trace3). 25M warm-up, 25M simulated.
  - SPEC CPU 2017 traces from the Third Data Prefetching Championship (DPC-3) set [6]. Screening: 25M warm-up, 25M simulated. Final set: 50M warm-up, 100M simulated.

4.2 Performance

Trace	IPC Discard	IPC Permit	Speedup (Permit / Discard)	STLB MPKI Discard	STLB MPKI Permit
Trace1	0.2335	0.2334	0.99996	2.855	2.856
Trace2	0.2513	0.2513	1.00002	2.726	2.727
Trace3	0.3829	0.3829	1.00004	0.934	0.937
Geomean			**1.00000**		

Figure 1: IPC, speedup (Permit IPC over Discard IPC) and STLB MPKI.

- Permit does not change IPC. The geomean speedup is 1.00000.
- STLB MPKI is almost the same. No visible STLB pollution.

4.3 Page-cross prefetches and translation cost

Trace	Page-cross requests (Permit)	STLB misses for prefetches (Permit)	% page-cross needing walk	Change in useful pf	Change in useless pf	Change in late pf
Trace1	33,740	122	0.36%	+510	+364	+133
Trace2	8,390	18	0.21%	+70	+57	+24
Trace3	16,303	82	0.50%	+38	+323	+89

Figure 2: Page-cross requests are requests into the L1D prefetch queue. STLB misses for prefetches are STLB misses for L1D prefetch translations.

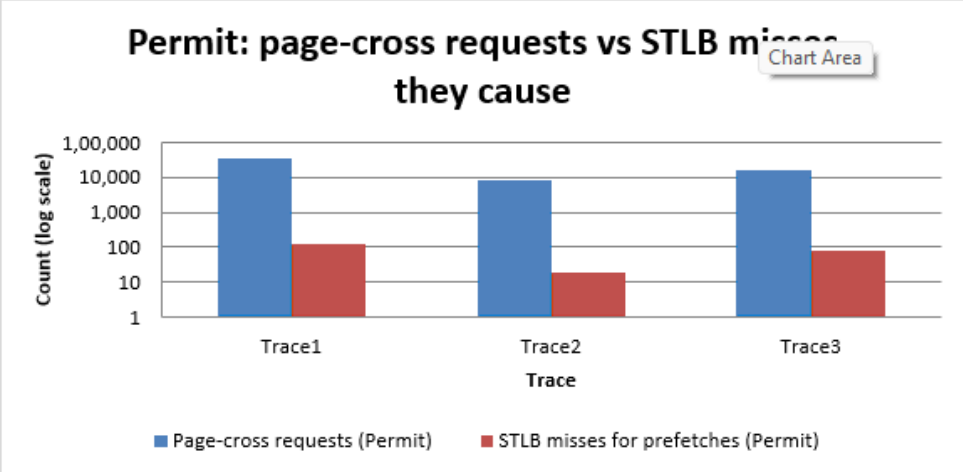


Figure 3: Page-cross prefetch requests and the STLB misses they cause (log scale).

- Page-cross requests are frequent: 8K to 34K per 25M instructions.
- Very few need a page walk: 0.2% to 0.5% of page-cross requests miss in the STLB.
- Few page-cross requests bring new lines. Most target lines already in the cache or already requested.
- New page-cross prefetches are split between useful and useless (Trace1, Trace2) or mostly useless (Trace3).

4.4 Degree sweep

- Question: is page-crossing cheap only because IP-stride prefetches 3 lines ahead?
- Change: `PREFETCH_DEGREE` set to 8, 16 and 32, in both Discard and Permit. Same traces and instruction counts.
- Degree 32 gives the same results as degree 16. On investigating, I realised the L1D prefetch queue has 16 entries (`L1D_PQ_SIZE`), so extra requests are refused.

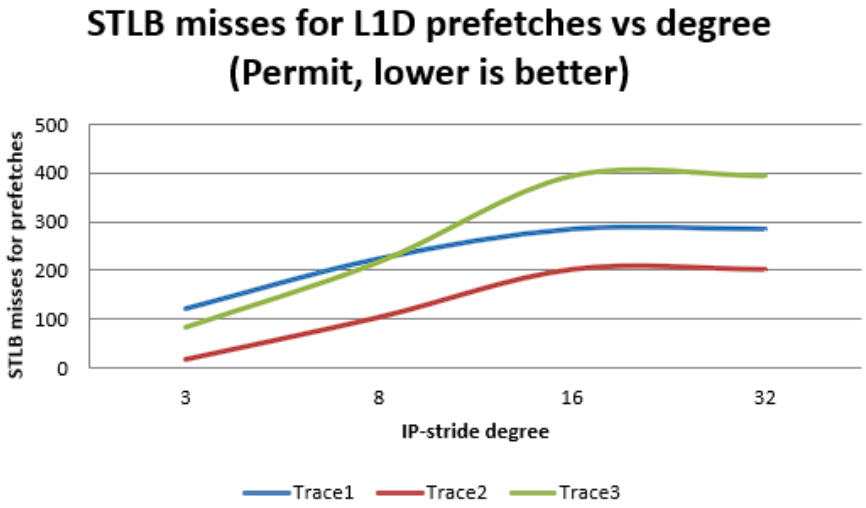


Figure 4: Speedup of Permit over Discard for each degree.

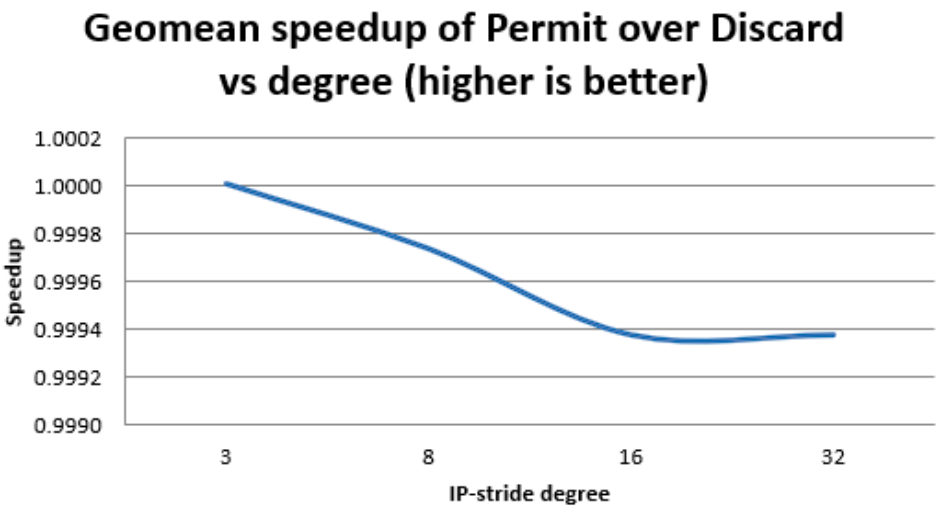


Figure 5: STLB misses for L1D prefetches in Permit mode for each degree.

- Translation cost depends on prefetch distance. The further ahead the prefetcher runs, the more often the next page is not yet translated.
- STLB pollution comes from the prefetch translations, not from prefetching itself: it appears only in Permit.
- The extra distance mostly adds useless page-cross prefetches, so the IP-stride pattern does not stay valid far beyond the page boundary. The loss from crossing grows with distance.

4.5 SPEC CPU 2017

Trace selection

- 12 traces screened at degree 3 and 16 (25M warm-up, 25M simulated).
- Dropped: roms and cactuBSSN. IP-stride issues almost no prefetches on them, so Discard and Permit give the same IPC.
- Final set of 6, run with 50M warm-up and 100M simulated at degree 16:
  - mcf-1554B, mcf-472B, fotonik3d: page-crossing helps in screening.
  - mcf-994B, omnetpp-874B, xalancbmk-10B: high TLB pressure (STLB MPKI 4 to 8).

Trace	IPC Discard	IPC Permit	Speedup (Permit / Discard)	STLB MPKI Discard	STLB MPKI Permit	STLB MPKI change	STLB misses for prefetches (Permit)	Page-cross requests (Permit)	Change in useful pf	Change in useless pf	Change in late pf
605.mcf_s-1554B	0.2581	0.2816	1.0910	0.080	0.080	0.0%	189	1,48,76,057	+5,41,992	+5,29,502	+15,551
605.mcf_s-472B	0.3528	0.3757	1.0650	1.297	1.296	-0.0%	1,07,430	43,98,148	+1,81,336	+1,68,069	+8,620
649.fotonik3d_s-1176B	1.0002	1.0034	1.0032	0.137	0.137	0.0%	0	48,92,149	+8,288	+4,192	+1,525
605.mcf_s-994B	0.2463	0.2459	0.9984	8.078	8.110	+0.4%	4,464	9,53,407	+4,508	+36,722	-26
620.omnetpp_s-874B	0.2319	0.2317	0.9991	4.280	4.440	+3.7%	16,137	1,00,895	+2,566	+44,366	+53
623.xalancbmk_s-10B	0.2803	0.2803	1.0000	4.814	4.822	+0.2%	17,721	4,12,496	+12,044	+1,08,475	+76
Geomean			**1.0254**								

Figure 6: Final set, IP-stride degree 16, 50M warm-up and 100M simulated instructions. Change columns are Permit minus Discard.

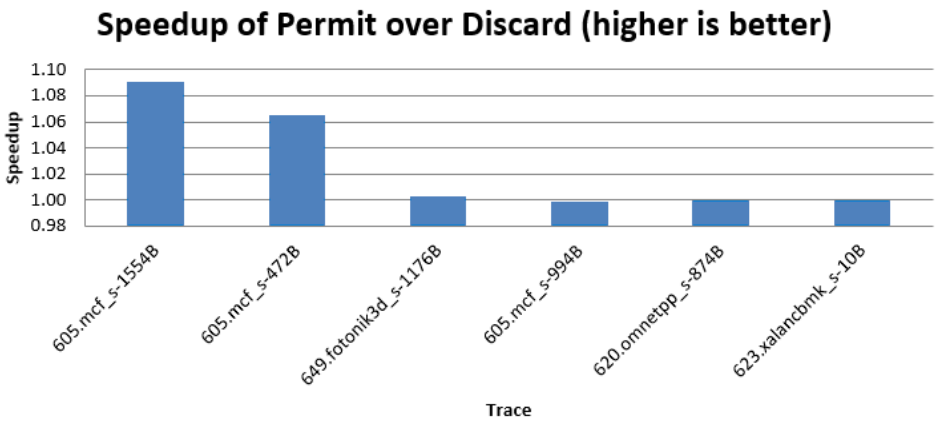


Figure 7: Speedup of Permit over Discard on the final set. Higher is better.

- On real workloads page-crossing can help a lot (up to 9.1% on mcf-1554B, geomean 1.025), unlike on the PA2 traces.
- Prediction quality decides whether crossing helps, not translation cost: about half of the new page-cross prefetches are useful where it helps, versus 8 to 17 useless per useful one where it hurts.
- STLB pollution appears only where crossing already hurts (up to 3.7% higher STLB MPKI on omnetpp-874B), so removing it can recover only a small loss (below 0.2%).
- mcf-472B gains 6.5% despite 107K page walks for prefetches. Section 4.6 shows these walks cost almost nothing.
- Some prefetch translations are later used by demand loads, so keeping them in a buffer is better than discarding them.
- The best prefetch distance is workload dependent (lbm: +0.18% at degree 3, -0.29% at degree 16).

4.6 Free translation (ideal case)

- Question: how much IPC does translation cost page-crossing?
- With `NO_TRANSLATION_PENALTY_FOR_PREFETCHES`, an L1D prefetch reads its translation from the page table directly: no TLB lookup, no page walk, no STLB fill. Prefetches to unmapped pages are dropped, as in normal mode.
- Same final set, degree 16, 50M warm-up, 100M simulated.

Trace	Speedup Permit/Discard (normal)	Speedup Permit/Discard (free	Permit IPC: free / normal	Discard IPC: free / normal	Page-cross requests (normal)	Page-cross requests (free)	Late pf Permit (normal)	Late pf Permit (free)	STLB MPKI Permit (normal)	STLB MPKI Permit (free)
605.mcf_s-1554B	1.0910	1.1273	1.0439	1.0103	1,48,76,057	1,77,25,177	2,51,609	1,61,556	0.080	0.080
605.mcf_s-472B	1.0650	1.0636	1.0008	1.0022	43,98,148	52,37,711	69,465	45,524	1.296	1.298
649.fotonik3d_s-1176B	1.0032	1.0000	0.9968	1.0000	48,92,149	59,46,826	10,728	8,796	0.137	0.137
605.mcf_s-994B	0.9984	0.9981	0.9998	1.0001	9,53,407	9,75,989	68	63	8.110	8.076
620.omnetpp_s-874B	0.9991	0.9989	0.9998	1.0000	1,00,895	1,07,925	95	106	4.440	4.280
623.xalancbmk_s-10B	1.0000	0.9981	0.9981	1.0000	4,12,496	4,75,887	822	926	4.822	4.814
Geomean	**1.0254**	**1.0299**								

Figure 8: Normal vs free translation for L1D prefetches.

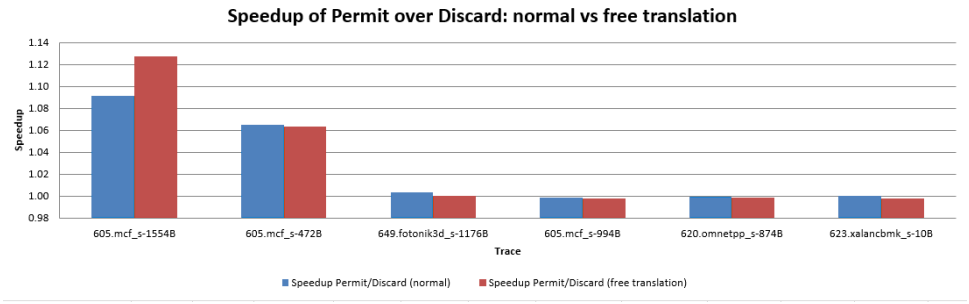


Figure 9: Speedup of Permit over Discard with normal and free translation. Higher is better.

- Translation limits page-crossing on mcf-1554B: free translation adds 4.4% to Permit (speedup over Discard 1.091 to 1.127), with only 189 page walks. The cost is translation latency, not page walks.
- Mechanism (mcf-1554B, Permit): prefetches wait in the 16-entry L1D prefetch queue for their STLB access. Queue occupancy is about 50% higher than with free translation, and 20.7M prefetch requests are refused for a full queue (15.5M with free translation).
- The page walks on mcf-472B and the STLB pollution on omnetpp-874B cost nothing measurable. Free translation removes both, and IPC does not change.
- Slow translation filters bad page-cross prefetches: on xalancbmk-10B, free translation lets more useless prefetches through and Permit loses 0.2%.

4.7 Observations

- PA2 traces: page-crossing is almost free and almost useless. Translation cost grows with prefetch distance but stays small (STLB MPKI below 3).
- SPEC CPU 2017: page-crossing can help a lot (up to 9.1%). Whether it helps depends mainly on how many page-cross prefetches are useful.
- The translation cost that matters is latency, not page walks or STLB pollution. Prefetches go to the STLB for every translation and hold prefetch queue entries while waiting.
- This changes the tPB target: a small buffer next to the L1D that gives prefetches the translations of the current and next page quickly, not a buffer that only avoids STLB pollution. The upper bound from free translation is 4.4% on mcf-1554B.

5 Next Steps

- Design the tPB as a small translation buffer at the L1D for prefetches (current and next page). Compare Discard, Permit and Permit with tPB against the free translation bound.
- Port Berti into the PA2 ChampSim and repeat Discard vs Permit.

6 Conclusion

- Done: code study of the translation path for L1D prefetches, Discard/Permit switch, degree sweep on the PA2 traces, screening of 12 SPEC CPU 2017 traces, long runs on a final set of 6, free translation runs.
- Not done yet: tPB, Berti.
- Takeaway: page-crossing helps or hurts mainly through prediction quality. Where it helps, translation latency (not page walks or pollution) limits the gain, up to 4.4% on mcf-1554B.
- Code and results: `https://github.com/sa1Lx/pagecross-data-prefetch`

References

[1] A. Navarro-Torres et al., "Berti: an Accurate Local-Delta Data Prefetcher," MICRO 2022.

[2] G. Vavouliotis et al., "To Cross, or Not to Cross Pages for Prefetching?," HPCA 2025.

[3] A. V. Jamet et al., "Enhancing Instruction Prefetching via Cache and TLB Management," arXiv:2605.12433, 2026.

[4] G. Vavouliotis et al., "Exploiting Page Table Locality for Agile TLB Prefetching," ISCA 2021.

[5] ChampSim simulator, CS683 PA2 version, `https://github.com/cs683-iitb-autumn-2026/pa2`.

[6] DPC-3 ChampSim traces, `https://dpc3.compas.cs.stonybrook.edu/champsim-traces/speccpu/`.